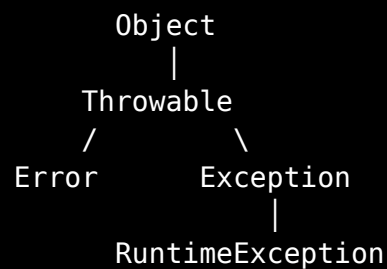


Understand Throwable, Error, Exception, and RuntimeException Hierarchy (Interview Notes)

Java Exception Hierarchy



- **Throwable** → Root class of all errors and exceptions.
 - **Error** → Serious system-level problems.
 - **Exception** → Conditions that applications can handle.
 - **RuntimeException** → Unchecked exceptions caused mainly by programming mistakes.
-

1. Throwable

What is Throwable?

Throwable is the **parent class** of all exceptions and errors in Java.

Only objects of Throwable (or its subclasses) can be thrown using the throw statement.

```
throw new Exception("Something went wrong");
```

2. Error

What is Error?

An Error represents **serious problems caused by the JVM or system**.

Applications are generally **not expected to catch or recover** from them.

Examples:

- OutOfMemoryError
 - StackOverflowError
 - VirtualMachineError
-

3. Exception

What is Exception?

An Exception represents a problem that **an application can handle**.

It allows the program to recover or continue execution.

Examples:

- IOException
 - SQLException
 - FileNotFoundException
-

4. RuntimeException

What is RuntimeException?

RuntimeException is a subclass of Exception.

These are **unchecked exceptions**, usually caused by programming mistakes.

Examples:

- NullPointerException
 - ArithmeticException
 - ArrayIndexOutOfBoundsException
 - ClassCastException
-

Checked vs Unchecked Exceptions

Checked Exception	Unchecked Exception
-------------------	---------------------

Checked at compile time	Checked at runtime
Must be handled or declared	Handling is optional
Extend Exception (excluding RuntimeException)	Extend RuntimeException
Example: IOException	Example: NullPointerException

Real-world Analogy

Imagine driving a car:

- **Error** → Engine completely destroyed (you cannot continue driving).
 - **Exception** → Flat tire (can be repaired and continue).
 - **RuntimeException** → Driver forgot to wear a seatbelt or drove carelessly (programming mistake).
-

Important Interview Points

- Throwable is the **root class** for all exceptions and errors.
 - Error represents **serious JVM/system failures** and is generally **not handled** by applications.
 - Exception represents **recoverable conditions** that applications are expected to handle.
 - RuntimeException represents **unchecked exceptions**, usually caused by coding errors.
 - All RuntimeExceptions are Exceptions, but **not all Exceptions are RuntimeExceptions**.
 - Only **checked exceptions** must be handled using try-catch or declared with throws.
-

Tricky Interview Questions

Q1. What is the parent class of Exception and Error?

Answer:

Throwable.

Q2. What is the difference between Error and Exception?

Answer:

- **Error** → Serious JVM/system problem, generally unrecoverable.
 - **Exception** → Application-level problem that can usually be handled.
-

Q3. Is RuntimeException a checked exception?

Answer: No.

It is an **unchecked exception**.

Q4. Why are RuntimeExceptions called unchecked?

Answer:

Because the compiler does **not require** them to be caught or declared.

Q5. Can we catch an Error?

Answer: Yes, technically.

```
catch (Error e) { }
```

But it is **strongly discouraged** because most Errors indicate problems from which the application cannot safely recover.

Q6. Is NullPointerException checked or unchecked?

Answer:

Unchecked (RuntimeException).

Q7. Which exceptions must be handled by the programmer?

Answer:

Checked exceptions must either be caught using try-catch or declared with throws.

Q8. Can we throw an Error manually?

Answer: Yes, but it is very rare and generally considered bad practice.

```
throw new OutOfMemoryError();
```

Interview Summary

- **Throwable is the root of Java's exception hierarchy.**
- Error represents **serious JVM or system failures** and is generally not recoverable.
- Exception represents **recoverable application-level problems.**
- RuntimeException represents **unchecked exceptions** caused mainly by programming errors.
- **Checked exceptions** must be handled or declared, while **unchecked exceptions** do not require compile-time handling.
- Understanding the hierarchy helps determine **which problems should be prevented, handled, or allowed to propagate.**

Interview One-Liner:

"Throwable is the parent of both Error and Exception; Error represents serious JVM failures, Exception represents recoverable application issues, and RuntimeException represents unchecked programming errors."